

What is feature selection?

Feature selection is the process of selecting the most relevant features of a [dataset](#) to use when building and training a [machine learning](#) (ML) model. By reducing the feature space to a selected subset, feature selection improves [AI model](#) performance while lowering its computational demands.

A "feature" refers to an individual measurable property or characteristic of a data point: a specific attribute of the data that helps describe the phenomenon being observed. A dataset about housing might have features such as “number of bedrooms” and “year of construction.”

Feature selection is part of the [feature engineering](#) process, in which data scientists prepare data and curate a feature set for [machine learning algorithms](#). Feature selection is the portion of feature engineering concerned with choosing the features to use for the model.

The benefits of feature selection

The feature selection process streamlines a model by identifying the most important, impactful and nonredundant features in the dataset. Reducing the number of features enhances model efficiency and boosts performance.

The benefits of feature selection include:

- **Better model performance:** Irrelevant features weaken model performance. Conversely, choosing the right set of features for a model makes it more accurate, more precise and gives it better recall. Data features affect how models configure their weights during training, which in turn drives performance. This differs from [hyperparameter tuning](#), which occurs before training.
- **Reduced overfitting:** [Overfitting](#) happens when a model cannot generalize past its training data. Removing redundant features decreases overfitting and makes a model better able to generalize to new data.
- **Shorter training times:** By focusing on a smaller subset of features, algorithms take less time to train. Model creators can test, validate and deploy their models faster with a smaller set of selected features.
- **Lower compute costs:** A smaller dataset made of the best features makes for simpler [predictive models](#) that occupy less storage space. Their computational requirements are lower than those of more complex models.
- **Greater interpretability:** [Explainable AI](#) is focused on creating models that humans can understand. As models grow more complex, it becomes increasingly difficult to [interpret](#) their results. Simpler models are easier to monitor and explain.
- **Smoother implementation:** Simpler, smaller models are easier to work with by developers when building AI apps, such as those used in [data visualization](#).
- **Dimensionality reduction:** With more input variables in play, data points grow more distant within the model space. High-dimensional data has more empty space, which makes it more difficult for the machine learning algorithm to identify patterns and make good predictions.

Collecting more data can mitigate the curse of dimensionality, but selecting the most important features is more feasible and cost-effective.

What are features?

A feature is a definable quality of the items in a dataset. Features are also known as variables because their values can change from one data point to the next, and attributes because they characterize the data points in the dataset. Different features characterize the data points in various ways.

Features can be independent variables, dependent variables that derive their value from independent variables or combined attributes that are compiled from multiple other features.

The goal of feature selection is to identify the most important input variables that the model can use to predict dependent variables. The target variable is the dependent variable that the model is charged with predicting.

For example, in a database of employees, input features can include age, location, salary, title, performance metrics and duration of employment. An employer can use these variables to generate a target combined attribute representing an employee's likelihood of leaving for a better offer. Then, the employer can determine how to encourage those employees to stay.

Features can be broadly categorized into numerical or categorical variables.

- **Numerical variables** are quantifiable, such as length, size, age and duration.
- **Categorical variables** are anything that is nonnumerical, such as name, job title and location.

Before feature selection takes place, the [feature extraction](#) process transforms raw data into numerical features that machine learning models can use. Feature extraction simplifies the data and reduces the compute requirements needed to process it.

Supervised feature selection methods

[Supervised learning](#) feature selection uses the target variable to determine the most important features. Because the data features are already identified, the task is about identifying which input variables most directly affect the target variable. Correlation is the primary criterion when assessing the most important features.

Supervised feature selection methods include:

- **Filter methods**
- **Wrapper methods**
- **Embedded methods**

Hybrid methods that combine two or more supervised feature selection methods are also possible.

Filter methods

Filter methods are a group of feature selection techniques that are solely concerned with the data itself and do not directly consider model performance optimization. Input variables are assessed independently against the target variable to determine which has the highest correlation. Methods that test features one by one are known as univariate feature selection methods.

Often used as a data preprocessing tool, filter methods are fast and efficient feature selection algorithms that excel at lowering redundancy and removing irrelevant features from the dataset. Various statistical tests are used to score each input variable for correlation. However, other methods are better at predicting model performance.

Available in popular machine learning libraries such as [Scikit-Learn \(Sklearn\)](#), some common filter methods are:

- **Information gain:** Measures how important the presence or absence of a feature is in determining the target variable by the degree of entropy reduction.
- **Mutual information:** Assesses the dependence between variables by measuring the information obtained about one through the other.
- **Chi-square test:** Assesses the relationship between two categorical variables by comparing observed to expected values.
- **Fisher's score:** Uses derivatives to calculate the relative importance of each feature for classifying data. A higher score indicates greater influence.
- **Pearson's correlation coefficient:** Quantifies the relationship between two continuous variables with a score ranging from -1 to 1.
- **Variance threshold:** Removes all features that fall under a minimum degree of variance because features with more variances are likely to contain more useful information. A related method is the mean absolute difference (MAD).
- **Missing value ratio:** Calculates the percentages of instances in a dataset for which a certain feature is missing or has a null value. If too many instances are missing a feature, it is not likely to be useful.
- **Dispersion ratio:** The ratio of variance to the mean value for a feature. Higher dispersion indicates more information.
- **ANOVA (analysis of variance):** Determines whether different feature values affect the value of the target variable.

Wrapper methods

Wrapper methods train the machine learning algorithm with various subsets of features, adding or removing features and testing the results at each iteration. The goal of all the wrapper methods is to find the feature set that leads to optimal model performance.

Wrapper methods that test all possible feature combinations are known as greedy algorithms. Their search for the overall best feature set is compute-intensive and time-consuming, and so is best for datasets with smaller feature spaces.

Data scientists can set the algorithm to stop when model performance decreases or when a target number of features is in play.

Wrapper methods include:

- **Forward selection:** Starts with an empty feature set and gradually adds new features until the optimal set is found. Model selection takes place when the algorithm's performance fails to improve after any specific iteration.
- **Backward selection:** Trains a model with all the original features and iteratively removes the least important feature from the feature set.
- **Exhaustive feature selection:** Tests every possible combination of features to find the overall best one by optimizing a specified performance metric. A logistic regression model that uses exhaustive feature selection tests every possible combination of every possible number of features.
- **Recursive feature elimination (RFE):** A type of backward selection that begins with an initial feature space and eliminates or adds features after each iteration based on their relative importance.
- **Recursive feature elimination with cross-validation:** A variation of recursive elimination that uses cross-validation, which tests a model on unseen data, to select the best-performing feature set. Cross-

validation is a common large language model ([LLM](#)) [evaluation](#) technique.

Embedded methods

Embedded methods fold—or embed—feature selection into the model training process. As the model undergoes training, it uses various mechanisms to detect underperforming features and discard those from future iterations.

Many embedded methods revolve around [regularization](#), which penalizes features based on a preset coefficient threshold. Models trade a degree of accuracy for greater precision. The result is that models perform slightly less well during training, but become more generalizable by reducing overfitting.

Embedded methods include:

- **[LASSO regression](#) (L1 regression):** Adds a penalty to the loss function for high-value correlated coefficients, moving them toward a value of 0. Coefficients with a value of 0 are removed. The greater the penalization, the more features are removed from the feature space. Effective LASSO use is about balancing the penalty to remove enough irrelevant features while keeping all the important ones.
- **[Random forest](#) importance:** Builds hundreds of [decision trees](#), each with a random selection of data points and features. Each tree is assessed by how well it divides the data points. The better the results, the more important the feature or features in that tree are considered to be. [Classifiers](#) measure the “impurity” of the groupings by Gini impurity or information gain, while regression models use variance.
- **[Gradient boosting](#):** Adds predictors in sequence to an ensemble with each iteration correcting the errors of the previous one. In this way, it can identify which features lead most directly to optimal results.

Unsupervised feature selection methods

With unsupervised learning, models figure out data features, patterns and relationships on their own. It’s not possible to tailor input variables to a known target variable. Unsupervised feature selection methods use other techniques to simplify and streamline the feature space.

One unsupervised feature selection method is [principal component analysis \(PCA\)](#). PCA reduces the dimensionality of large datasets by transforming potentially correlated variables into a smaller set of variables. These principal components retain most of the information contained in the original dataset. PCA counters the curse of dimensionality and also reduces overfitting.

Others include independent component analysis (ICA), which separates multivariate data into individual components that are statistically independent, and [autoencoders](#).

Widely used with [transformer](#) architectures, an autoencoder is a type of [neural network](#) that learns to compress and then reconstruct data. In doing so, autoencoders discover latent variables—those which are not directly observable, but that strongly affect data distribution.

Choosing a feature selection method

The type of feature selection used depends on the nature of the input and output variables. These also shape the nature of the machine learning challenge—whether it’s a classification problem or a regression task.

- **Numerical input, numerical output:** When inputs and outputs are both numerical, this indicates a regression predictive problem. [Linear models](#) output for continuous numerical predictions—outputting a target variable that is a number within a range of possible values. In these cases, correlation coefficients, such as Pearson’s correlation coefficient, are an ideal feature selection method.

- **Numerical input, categorical output:** [Logistic regression](#) models classify inputs into discrete categorical outputs. In this classification problem, correlation-based feature selection methods that support categorical target variables can be used. These include ANOVA for linear regression models and Kendall's coefficient of rank correlation for nonlinear tasks.
- **Categorical input, numerical output:** This rare type of challenge can also be solved with correlation methods that support categorical variables.
- **Categorical input, categorical output:** Classification problems with categorical input and target variables lend themselves to the chi-squared method or information gain techniques.

Other factors to consider include the size of the dataset and feature space, feature complexity and model type. Filter methods can quickly eliminate a large portion of irrelevant features, but struggle with complex feature interactions. In these cases, wrapper and embedded methods might be more suitable.

What makes features important?

Knowing which features to focus on is the essential component of feature selection. Some features are highly desirable for modeling, while others can lead to subpar results. In addition to how they affect target variables, feature importance is determined by:

- **Ease of modeling:** If a feature is easy to model, the overall machine learning process is simpler and faster, with fewer opportunities for error.
- **Easy to regularize:** Features that take well to regularization will be more efficient to work with.
- **Disentangling causality:** Disentangling causal factors from an observable feature means identifying the underlying factors that influence it.